Name :- Jai Pratap Rajput

CU ID : CU24250071

BTech CSE 3rd Year SECTION - "C"

Roll No. :- 27

==========

Lab Sheet No. 5

LAB SHEET 5

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
import ipywidgets as widgets

from IPython.display import display

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
from google.colab import files

uploaded = files.upload()

file_name = list(uploaded.keys())[0]

df = pd.read_csv(file_name)

print("Dataset loaded successfully!")
print("File:", file_name)
```

```
Choose Files  Superstore…Dataset.csv
Superstore_Sales_Dataset.csv(text/csv) - 987113 bytes, last modified: 9/11/2026 - 100% done
Saving Superstore_Sales_Dataset.csv to Superstore_Sales_Dataset (1).csv
Dataset loaded successfully!
File: Superstore_Sales_Dataset (1).csv
```

```
print("First 5 rows of the dataset:")

display(df.head())
```

```
First 5 rows of the dataset:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity
0	1	ORD-002023	2023-08-02	2023-08-04	Same Day	CUST-1139	Customer 787	Consumer	United States	Philadelphia	...	37122	East	PROD-0710	Office Supplies	Art	Art Product 15	320.36	
1	2	ORD-002024	2025-11-30	2025-12-06	Standard Class	CUST-0610	Customer 658	Consumer	United States	Richmond	...	98017	South	PROD-0774	Office Supplies	Art	Art Product 52	177.44	
2	3	ORD-002025	2023-05-19	2023-05-23	Second Class	CUST-1098	Customer 1126	Home Office	United States	Cleveland	...	55975	Central	PROD-0161	Technology	Phones	Phones Product 19	245.62	
3	4	ORD-002026	2024-01-10	2024-01-15	Second Class	CUST-0307	Customer 675	Consumer	United States	Columbus	...	36691	Central	PROD-0107	Office Supplies	Paper	Paper Product 16	147.25	

```
print("Dataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)
```

```
Dataset Shape:
(5000, 21)

Column Names:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Pro

Data Types:
Row ID            int64
Order ID         object
Order Date       object
Ship Date        object
Ship Mode        object
Customer ID      object
Customer Name    object
Segment          object
Country          object
City             object
State            object
Postal Code       int64
Region           object
Product ID       object
Category         object
Sub-Category     object
Product Name     object
Sales           float64
Quantity          int64
Discount        float64
Profit          float64
dtype: object
```

```
print("Missing Values in Each Column:")

print(df.isnull().sum())
```

```
Missing Values in Each Column:
Row ID          0
Order ID        0
Order Date      0
Ship Date       0
```

```
Ship Mode       0
Customer ID     0
Customer Name   0
Segment         0
Country         0
City            0
State           0
Postal Code     0
Region          0
Product ID      0
Category        0
Sub-Category    0
Product Name    0
Sales           0
Quantity        0
Discount        0
Profit          0
dtype: int64
```

```
print("Rows before removing duplicates:", len(df))

df = df.drop_duplicates()

print("Rows after removing duplicates:", len(df))
```

```
Rows before removing duplicates: 5000
Rows after removing duplicates: 5000
```

```
df['Order Date'] = pd.to_datetime(df['Order Date'])
df['Ship Date'] = pd.to_datetime(df['Ship Date'])

print("Date columns converted successfully!")

print(df[['Order Date', 'Ship Date']].head())
```

```
Date columns converted successfully!
  Order Date  Ship Date
0 2023-08-02 2023-08-04
1 2025-11-30 2025-12-06
2 2023-05-19 2023-05-23
3 2024-01-10 2024-01-15
4 2023-10-01 2023-10-04
```

```
print("Statistical Summary:")

display(df.describe())
```

Statistical Summary:

	Row ID	Order Date	Ship Date	Postal Code	Sales	Quantity	Discount	Profit
count	5000.000000	5000	5000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000
mean	2500.500000	2024-06-25 20:24:51.840000	2024-06-29 20:59:25.440000	54765.013800	244.581150	4.006000	0.155280	22.114000
min	1.000000	2023-01-01 00:00:00	2023-01-04 00:00:00	10009.000000	8.000000	1.000000	0.000000	-205.110000
25%	1250.750000	2023-09-19 18:00:00	2023-09-24 00:00:00	31876.250000	89.962500	2.000000	0.050000	4.280000
50%	2500.500000	2024-06-24 00:00:00	2024-06-27 12:00:00	54770.000000	163.305000	4.000000	0.150000	12.055000
75%	3750.250000	2025-03-31 06:00:00	2025-04-04 06:00:00	77177.250000	301.185000	6.000000	0.200000	27.950000
max	5000.000000	2025-12-30 00:00:00	2026-01-04 00:00:00	99997.000000	4585.550000	7.000000	0.400000	922.250000
std	1443.520003	NaN	NaN	25889.585644	264.380099	1.992777	0.115615	38.503405

```
total_sales = df['Sales'].sum()

total_profit = df['Profit'].sum()

total_orders = df['Order ID'].nunique()

average_order_value = total_sales / total_orders

customer_count = df['Customer ID'].nunique()

print("======================================")
print("           BUSINESS KPIs")
print("======================================")

print(f"Total Sales         : ${total_sales:,.2f}")
print(f"Total Profit        : ${total_profit:,.2f}")
print(f"Total Orders        : {total_orders:,}")
print(f"Average Order Value : ${average_order_value:,.2f}")
print(f"Customer Count      : {customer_count:,}")

print("======================================")
```

```
======================================
           BUSINESS KPIs
======================================
Total Sales         : $1,222,905.75
Total Profit        : $110,570.00
Total Orders        : 5,000
Average Order Value : $244.58
Customer Count      : 1,187
======================================
```

```
fig = go.Figure()

fig.add_trace(go.Indicator(
    mode="number",
    value=total_sales,
    title={"text": "Total Sales"},
    number={"prefix": "$", "valueformat": ",.0f"},
    domain={'x': [0, 0.2], 'y': [0, 1]}
))
```

```
fig.add_trace(go.Indicator(
    mode="number",
    value=total_profit,
    title={"text": "Total Profit"},
    number={"prefix": "$", "valueformat": ",.0f"},
    domain={'x': [0.2, 0.4], 'y': [0, 1]}
))

fig.add_trace(go.Indicator(
    mode="number",
    value=total_orders,
    title={"text": "Total Orders"},
    number={"valueformat": ","},
    domain={'x': [0.4, 0.6], 'y': [0, 1]}
))

fig.add_trace(go.Indicator(
    mode="number",
    value=average_order_value,
    title={"text": "Average Order Value"},
    number={"prefix": "$", "valueformat": ",.2f"},
    domain={'x': [0.6, 0.8], 'y': [0, 1]}
))

fig.add_trace(go.Indicator(
    mode="number",
    value=customer_count,
    title={"text": "Customers"},
    number={"valueformat": ","},
    domain={'x': [0.8, 1], 'y': [0, 1]}
))

fig.update_layout(
    title="Business Performance KPIs",
    height=300
)

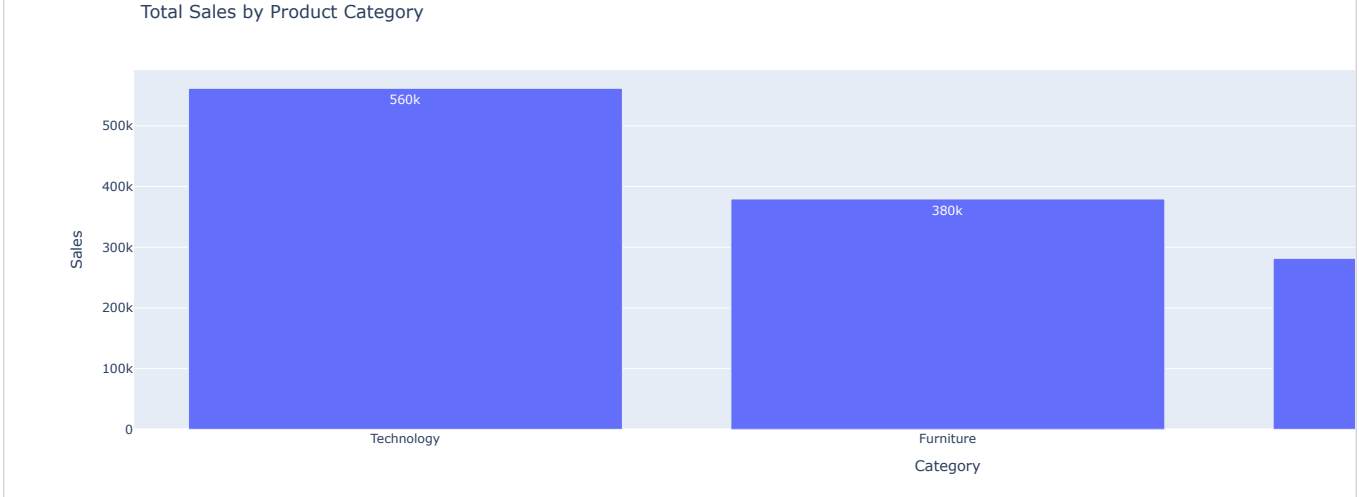
fig.show()
```




```
category_sales = (
    df.groupby('Category')['Sales']
    .sum()
    .reset_index()
    .sort_values('Sales', ascending=False)
)

fig = px.bar(
    category_sales,
    x='Category',
    y='Sales',
    title='Total Sales by Product Category',
    text_auto='.2s'
)

fig.show()
```

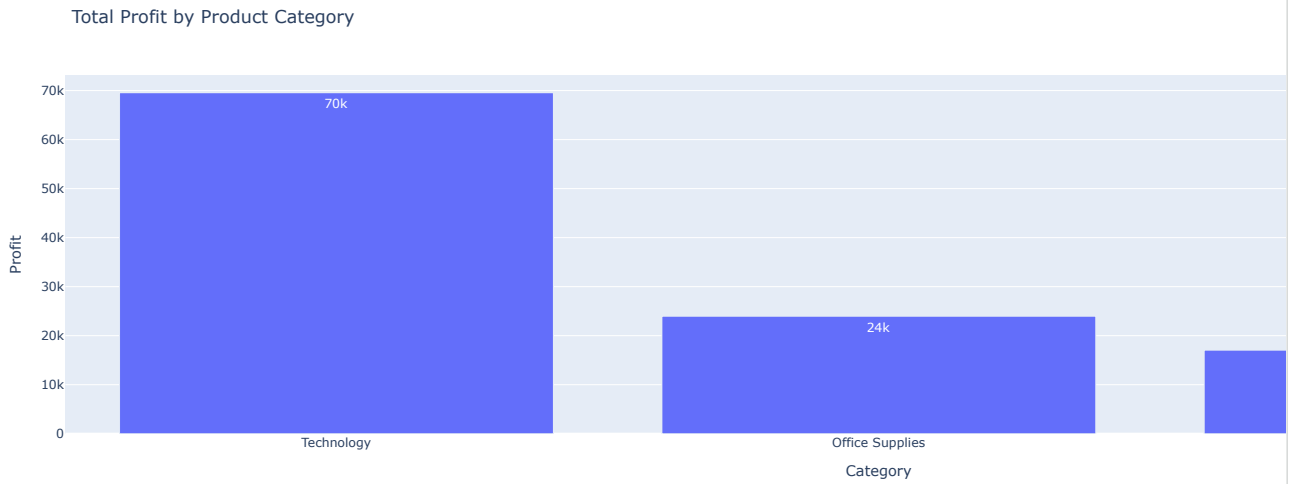



```
category_profit = (
    df.groupby('Category')['Profit']
    .sum()
```

```
    .reset_index()
    .sort_values('Profit', ascending=False)
)

fig = px.bar(
    category_profit,
    x='Category',
    y='Profit',
    title='Total Profit by Product Category',
    text_auto='.2s'
)

fig.show()
```



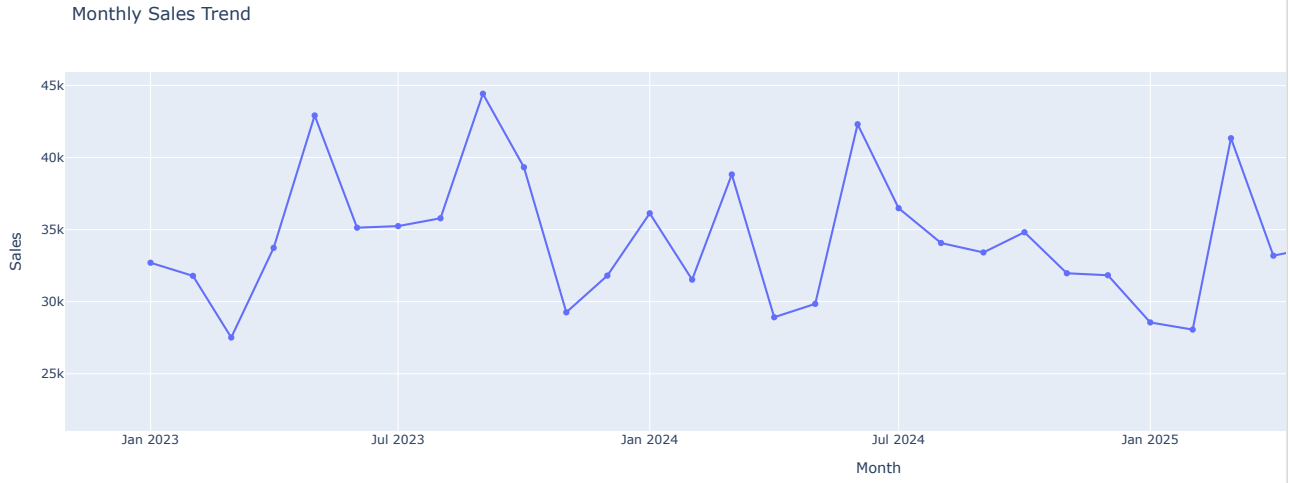

```
monthly_sales = (
    df.groupby(df['Order Date'].dt.to_period('M'))['Sales']
    .sum()
    .reset_index()
)

monthly_sales['Order Date'] = monthly_sales['Order Date'].astype(str)

fig = px.line(
    monthly_sales,
    x='Order Date',
    y='Sales',
    title='Monthly Sales Trend',
    markers=True
)

fig.update_layout(
    xaxis_title='Month',
    yaxis_title='Sales'
)

fig.show()
```

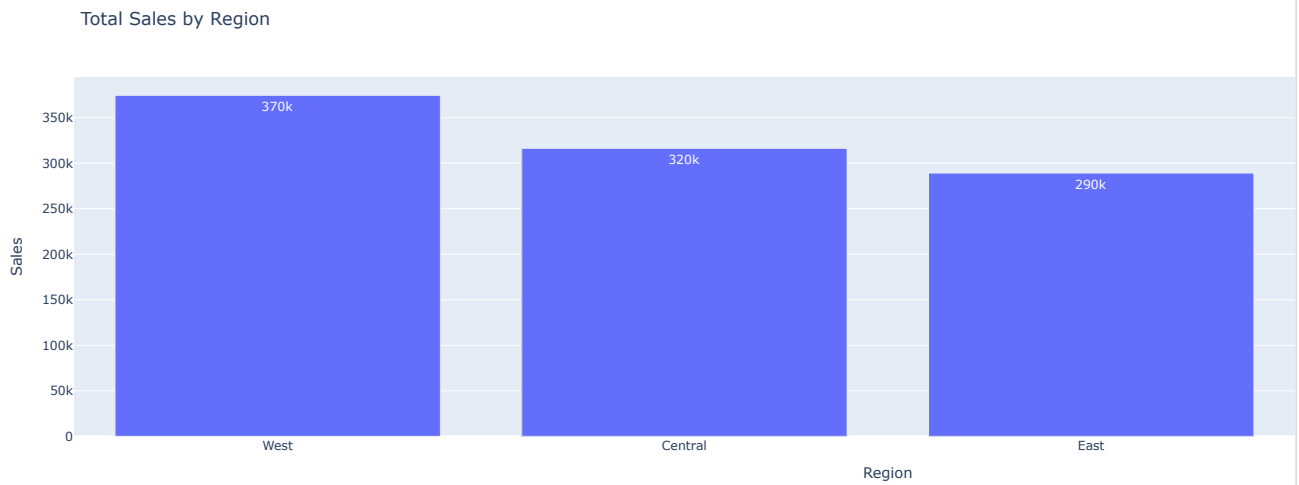



```
region_sales = (
    df.groupby('Region')['Sales']
    .sum()
    .reset_index()
    .sort_values('Sales', ascending=False)
)

fig = px.bar(
    region_sales,
    x='Region',
```

```
    y='Sales',
    title='Total Sales by Region',
    text_auto='.2s'
)

fig.show()
```




```
import ipywidgets as widgets

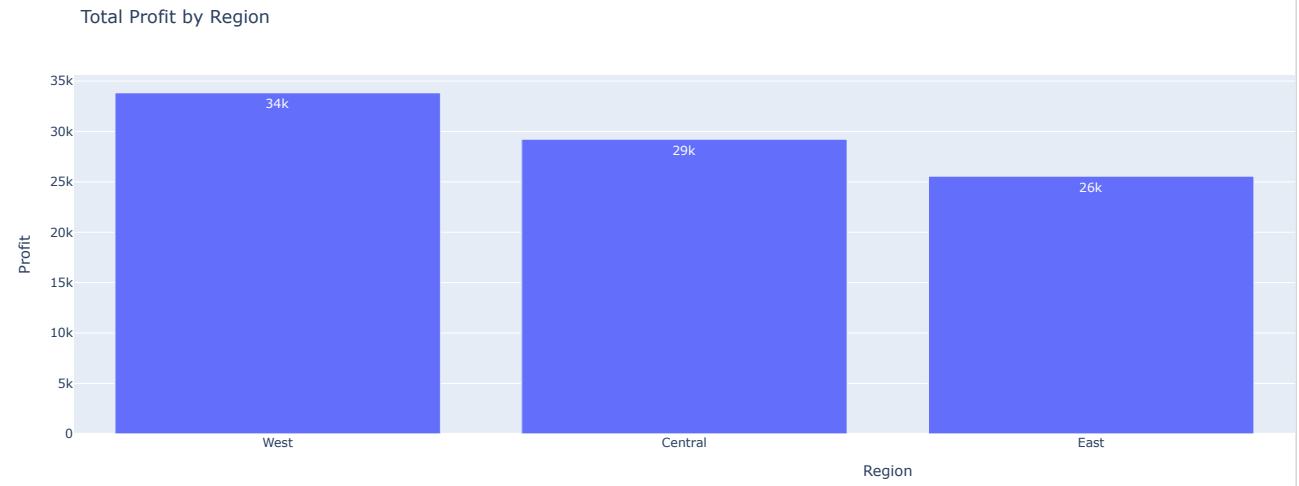
print("ipywidgets imported successfully!")
```

```
ipywidgets imported successfully!
```

```
region_profit = (
    df.groupby('Region')['Profit']
    .sum()
    .reset_index()
    .sort_values('Profit', ascending=False)
)

fig = px.bar(
    region_profit,
    x='Region',
    y='Profit',
    title='Total Profit by Region',
    text_auto='.2s'
)

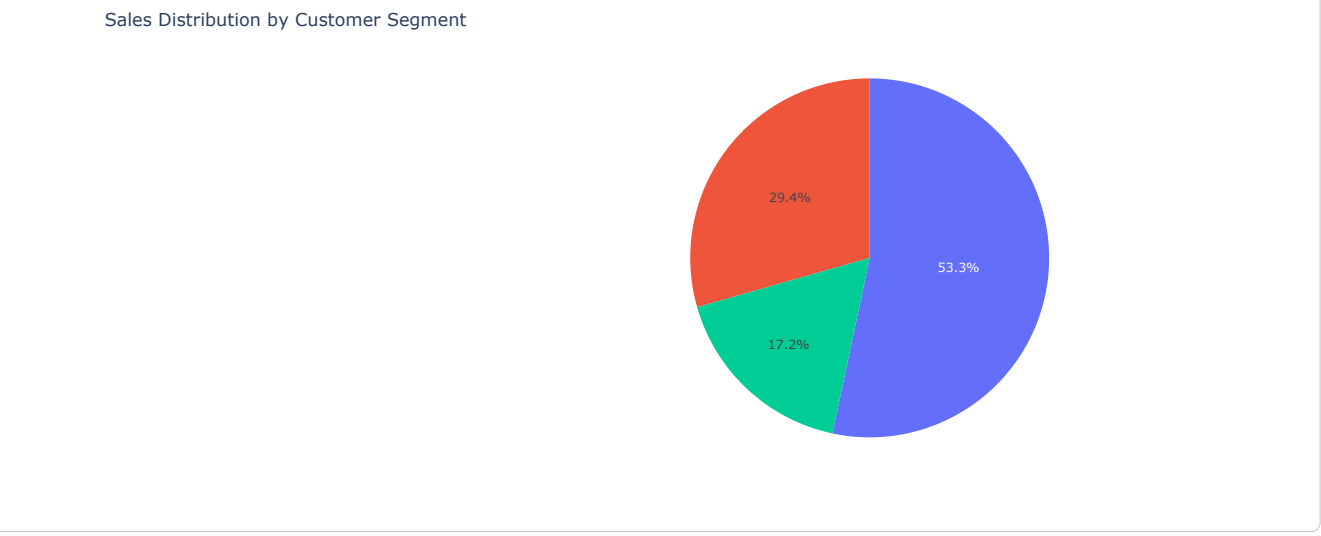
fig.show()
```




```
segment_sales = (
    df.groupby('Segment')['Sales']
    .sum()
    .reset_index()
)

fig = px.pie(
    segment_sales,
    names='Segment',
    values='Sales',
    title='Sales Distribution by Customer Segment'
)

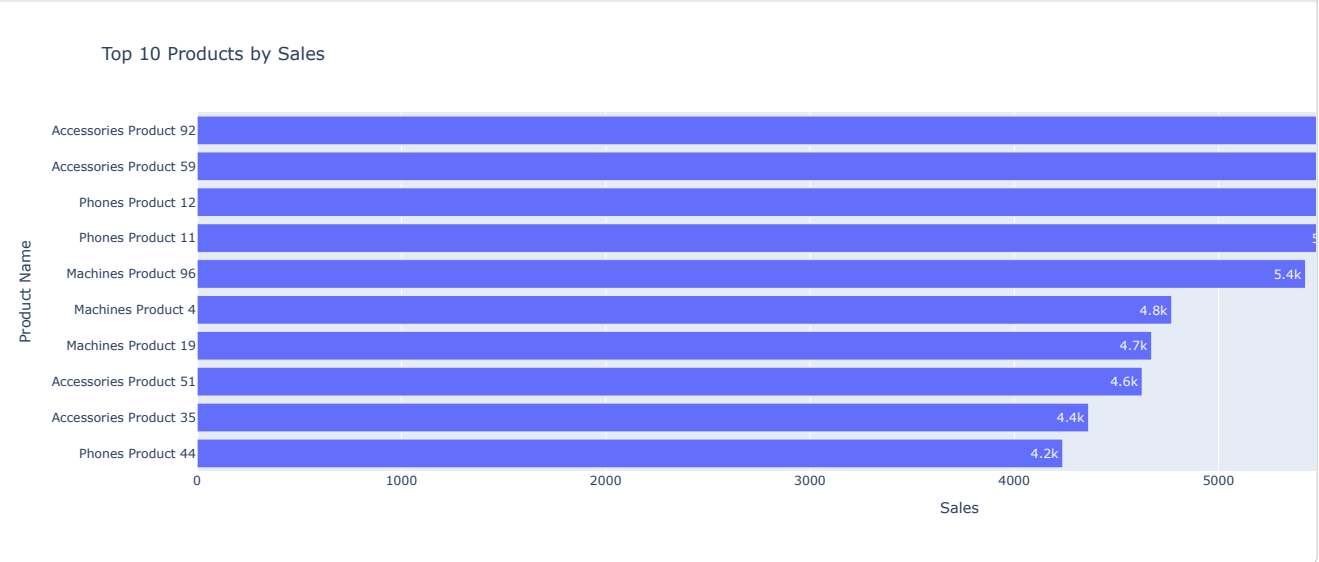
fig.show()
```




```
top_products = (
    df.groupby('Product Name')['Sales']
    .sum()
    .reset_index()
    .sort_values('Sales', ascending=False)
    .head(10)
)

fig = px.bar(
    top_products.sort_values('Sales'),
    x='Sales',
    y='Product Name',
    orientation='h',
    title='Top 10 Products by Sales',
    text_auto='.2s'
)

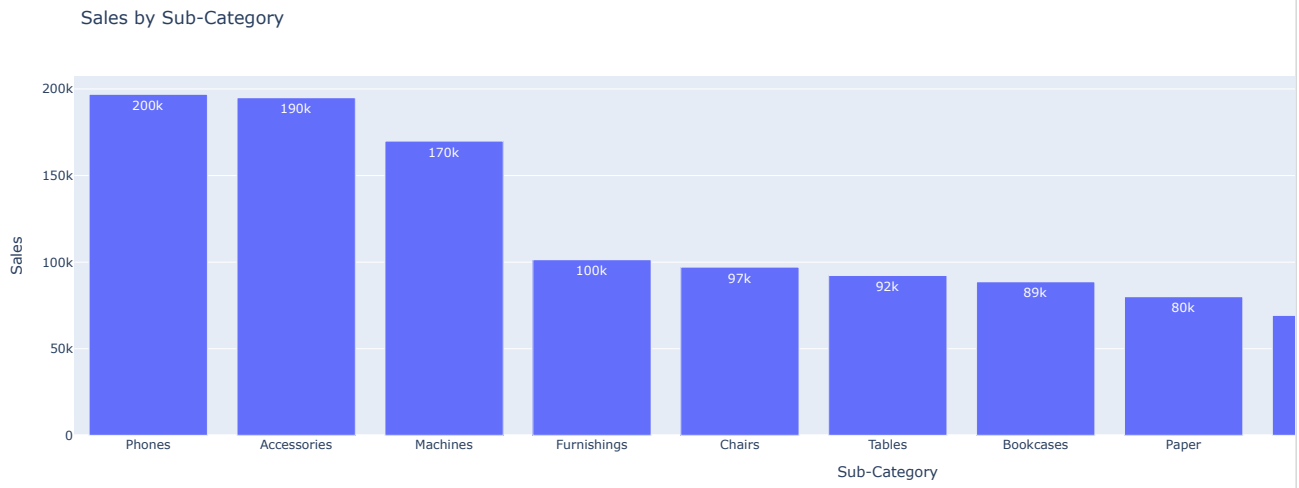
fig.show()
```




```
subcategory_sales = (
    df.groupby('Sub-Category')['Sales']
    .sum()
    .reset_index()
    .sort_values('Sales', ascending=False)
)

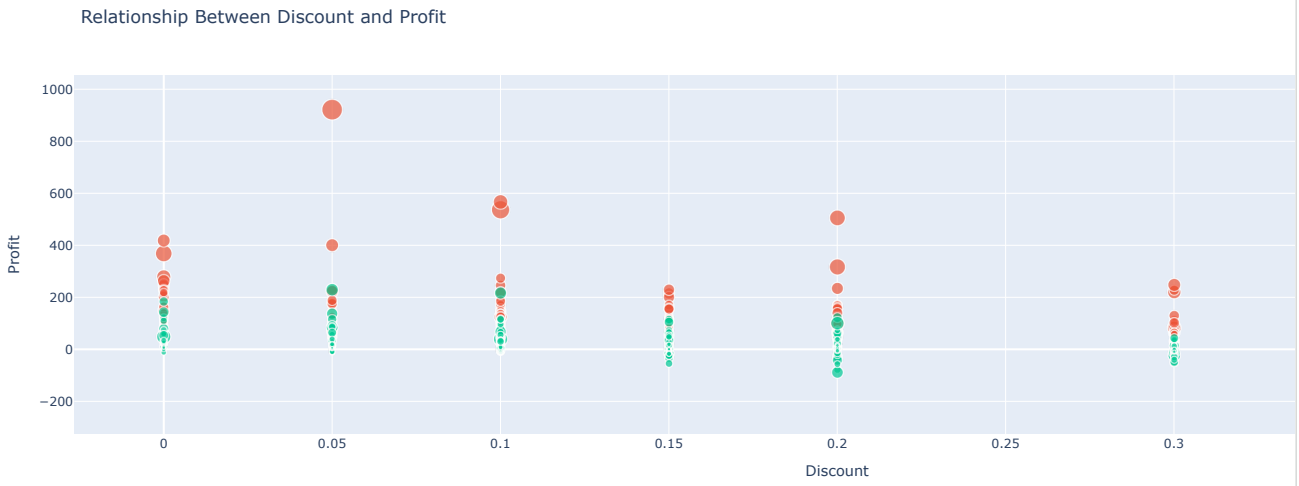
fig = px.bar(
    subcategory_sales,
    x='Sub-Category',
    y='Sales',
    title='Sales by Sub-Category',
    text_auto='.2s'
)

fig.show()
```




```
fig = px.scatter(
    df,
    x='Discount',
    y='Profit',
    size='Sales',
    color='Category',
    hover_data=['Product Name', 'Region'],
    title='Relationship Between Discount and Profit'
)

fig.show()
```




```
categories = ['All'] + sorted(df['Category'].unique())

category_dropdown = widgets.Dropdown(
    options=categories,
    description='Category:',
    style={'description_width': 'initial'}
)

output = widgets.Output()

def category_dashboard(change=None):

    with output:
        output.clear_output(wait=True)

        selected_category = category_dropdown.value

        if selected_category == 'All':
            filtered_df = df.copy()
        else:
            filtered_df = df[
                df['Category'] == selected_category
            ]

        sales = filtered_df['Sales'].sum()
        profit = filtered_df['Profit'].sum()
        orders = filtered_df['Order ID'].nunique()

        print("====================================")
        print("       FILTERED DASHBOARD")
        print("====================================")

        print(f"Category     : {selected_category}")
        print(f"Sales        : ${sales:,.2f}")
        print(f"Profit       : ${profit:,.2f}")
```

```
        print(f"Orders       : {orders:,}")

        region_data = (
            filtered_df.groupby('Region')['Sales']
            .sum()
            .reset_index()
        )

        fig = px.bar(
            region_data,
            x='Region',
            y='Sales',
            title=f'Sales by Region — {selected_category}',
            text_auto='.2s'
        )

        fig.show()


category_dropdown.observe(
    category_dashboard,
    names='value'
)

display(category_dropdown)
display(output)

category_dashboard()
```

Category: All

```
==================================
       FILTERED DASHBOARD
==================================
Category     : All
Sales        : $1,222,905.75
Profit       : $110,570.00
Orders       : 5,000
```

```
regions = ['All'] + sorted(df['Region'].unique())

region_dropdown = widgets.Dropdown(
    options=regions,
    description='Region:',
    style={'description_width': 'initial'}
)

output_region = widgets.Output()

def region_dashboard(change=None):

    with output_region:
        output_region.clear_output(wait=True)

        selected_region = region_dropdown.value

        if selected_region == 'All':
            filtered_df = df.copy()
        else:
            filtered_df = df[
                df['Region'] == selected_region
            ]

        sales = filtered_df['Sales'].sum()
        profit = filtered_df['Profit'].sum()
        orders = filtered_df['Order ID'].nunique()

        print("==================================")
        print("       REGION DASHBOARD")
        print("==================================")

        print(f"Region  : {selected_region}")
        print(f"Sales   : ${sales:,.2f}")
        print(f"Profit  : ${profit:,.2f}")
        print(f"Orders  : {orders:,}")

        category_data = (
            filtered_df.groupby('Category')['Sales']
            .sum()
```

```
            .reset_index()
        )

        fig = px.bar(
            category_data,
            x='Category',
            y='Sales',
            title=f'Sales by Category — {selected_region}',
            text_auto='.2s'
        )

        fig.show()


region_dropdown.observe(
    region_dashboard,
    names='value'
)

display(region_dropdown)
display(output_region)

region_dashboard()
```

```
Region: All
====================================
       REGION DASHBOARD
====================================
Region  : All
Sales   : $1,222,905.75
Profit  : $110,570.00
Orders  : 5,000
```

```
category_filter = widgets.Dropdown(
    options=['All'] + sorted(df['Category'].unique()),
    description='Category:'
)

region_filter = widgets.Dropdown(
    options=['All'] + sorted(df['Region'].unique()),
    description='Region:'
)

segment_filter = widgets.Dropdown(
    options=['All'] + sorted(df['Segment'].unique()),
    description='Segment:'
)

dashboard_output = widgets.Output()


def update_full_dashboard(change=None):

    with dashboard_output:

        dashboard_output.clear_output(wait=True)

        filtered = df.copy()

        if category_filter.value != 'All':
            filtered = filtered[
                filtered['Category'] == category_filter.value
            ]

        if region_filter.value != 'All':
            filtered = filtered[
                filtered['Region'] == region_filter.value
            ]

        if segment_filter.value != 'All':
            filtered = filtered[
                filtered['Segment'] == segment_filter.value
            ]

        sales = filtered['Sales'].sum()
        profit = filtered['Profit'].sum()
        orders = filtered['Order ID'].nunique()
```

```
        print("========================================")
        print("       INTERACTIVE BUSINESS DASHBOARD")
        print("========================================")

        print(f"Sales  : ${sales:,.2f}")
        print(f"Profit : ${profit:,.2f}")
        print(f"Orders : {orders:,}")

        chart_data = (
            filtered.groupby('Category')['Sales']
            .sum()
            .reset_index()
        )

        fig = px.bar(
            chart_data,
            x='Category',
            y='Sales',
            title='Filtered Sales by Category',
            text_auto='.2s'
        )

        fig.show()


category_filter.observe(
    update_full_dashboard,
    names='value'
)

region_filter.observe(
    update_full_dashboard,
    names='value'
)

segment_filter.observe(
    update_full_dashboard,
    names='value'
)

display(category_filter)
display(region_filter)
display(segment_filter)

display(dashboard_output)

update_full_dashboard()
```

```
Category: All
Region: All
Segment: All
========================================
       INTERACTIVE BUSINESS DASHBOARD
========================================
Sales  : $1,222,905.75
Profit : $110,570.00
Orders : 5,000
```

```
best_category = (
    df.groupby('Category')['Sales']
    .sum()
    .idxmax()
)

best_category_sales = (
    df.groupby('Category')['Sales']
    .sum()
    .max()
)

print("Best Performing Category:")
print(best_category)

print(f"Sales: ${best_category_sales:,.2f}")
```

```
Best Performing Category:
Technology
Sales: $561,756.25
```

```
best_region = (
    df.groupby('Region')['Sales']
    .sum()
    .idxmax()
)

best_region_sales = (
    df.groupby('Region')['Sales']
    .sum()
    .max()
)

print("Highest Revenue Region:")
print(best_region)

print(f"Sales: ${best_region_sales:,.2f}")
```

```
Highest Revenue Region:
West
Sales: $374,489.38
```

```
best_product = (
    df.groupby('Product Name')['Sales']
    .sum()
    .idxmax()
)

best_product_sales = (
    df.groupby('Product Name')['Sales']
    .sum()
    .max()
)

print("Top Performing Product:")
print(best_product)

print(f"Sales: ${best_product_sales:,.2f}")
```

```
Top Performing Product:
Accessories Product 92
Sales: $7,092.76
```

```
print("================================================")
print("             BUSINESS INSIGHTS")
print("================================================")

print(f"""
1. The highest-selling product category is {best_category},
   with total sales of ${best_category_sales:,.2f}.

2. The highest-revenue region is {best_region},
   with total sales of ${best_region_sales:,.2f}.

3. The top-performing product is {best_product},
   generating sales of ${best_product_sales:,.2f}.

4. The dashboard provides a clear view of sales,
   profit, orders, customers and product performance.

5. Interactive filters allow users to analyze the
   business by category, region and customer segment.

6. Sales trends can be analyzed over time to identify
   periods of high and low business performance.

7. Profit analysis helps identify profitable and
   less-profitable product categories and regions.
""")
```

```
================================================
             BUSINESS INSIGHTS
================================================

1. The highest-selling product category is Technology,
   with total sales of $561,756.25.

2. The highest-revenue region is West,
   with total sales of $374,489.38.

3. The top-performing product is Accessories Product 92,
   generating sales of $7,092.76.

4. The dashboard provides a clear view of sales,
   profit, orders, customers and product performance.

5. Interactive filters allow users to analyze the
   business by category, region and customer segment.

6. Sales trends can be analyzed over time to identify
   periods of high and low business performance.

7. Profit analysis helps identify profitable and
   less-profitable product categories and regions.
```

Q1. What is Business Intelligence (BI)? How does it support organizational decision-making?

Ans - Business Intelligence (BI) is a technology-driven process used to collect, analyze, and present business data in a meaningful form. It converts raw data into useful information through reports, dashboards, charts, and visualizations. BI supports organizational decision-making by helping managers identify trends, measure performance, compare business results, and make accurate data-driven decisions instead of relying on assumptions.

Q2. What is an interactive dashboard? How is it different from a static report?

Ans - An interactive dashboard is a visual display of important business information that allows users to interact with data using filters, slicers, drill-down options, and dynamic charts. A static report only presents fixed information and cannot be changed by the user. Interactive dashboards improve data exploration by allowing users to analyze different regions, categories, customers, or time periods instantly.

Q3. Compare Tableau and Microsoft Power BI based on features, usability, and business applications.

Ans - Tableau and Microsoft Power BI are popular Business Intelligence tools used for data visualization and dashboard development. Tableau is well known for advanced visual analytics and highly interactive visualizations, while Power BI is widely used because of its strong integration with Microsoft Office and easy report creation. Tableau is often preferred for detailed visual exploration, whereas Power BI is commonly used in organizations for business reporting, KPI monitoring, and enterprise analytics.

Q4. What are Key Performance Indicators (KPIs)? Give examples of KPIs used in sales analytics.

Ans - Key Performance Indicators (KPIs) are measurable values used to evaluate the success and performance of a business or organization. They help managers monitor whether business objectives are being achieved. Common sales KPIs include Total Sales, Total Profit, Number of Orders, Average Order Value, Customer Count, Revenue Growth, and Profit Margin. These indicators provide a quick overview of business performance.

Q5. Explain the purpose of filters, slicers, and drill-down functionality in dashboards.

Ans - Filters, slicers, and drill-down features make dashboards interactive and user-friendly. Filters allow users to display specific data based on selected conditions such as region or category. Slicers provide visual filtering controls that simplify data selection. Drill-down functionality enables users to move from summarized information to detailed data, helping them investigate business performance at different levels.

Q6. What factors should be considered while designing an effective business dashboard?

Ans - An effective business dashboard should have a clear layout, meaningful titles, readable charts, accurate KPIs, appropriate colors, and well-organized visualizations. It should avoid unnecessary complexity and present important information in a simple and understandable manner. The dashboard should also include interactive features, proper formatting, and a logical arrangement so users can quickly understand business performance.

Q7. How can dashboards help managers monitor organizational performance in real time?

Ans - Dashboards help managers continuously monitor important business metrics by displaying updated sales, profit, customer, and operational information in one place. They allow managers to quickly identify positive trends, declining performance, high-performing regions, and areas requiring improvement. This enables faster responses and supports timely business decisions based on current data.

Q8. Why is data visualization important in Business Intelligence tools?

Ans - Data visualization is important because it transforms complex numerical data into easy-to-understand charts, graphs, maps, and dashboards. Visual representation helps users recognize patterns, trends, comparisons, and relationships much faster than raw tables. In Business Intelligence, effective visualization improves communication, simplifies analysis, and supports better decision-making across different organizational levels.

Q9. What insights can be derived from a sales dashboard for an e-commerce company?

Ans - A sales dashboard can provide valuable insights such as total revenue, total profit, monthly sales trends, top-selling products, customer purchasing behavior, high-performing regions, product category performance, and customer segment analysis. It also helps identify low-performing products or regions that require improvement, allowing businesses to optimize sales strategies and increase profitability.

Q10. Explain how interactive dashboards improve communication between data analysts and business stakeholders.

Ans - Interactive dashboards improve communication by presenting business data in a visual and understandable format that both technical and non-technical users can interpret easily. Data analysts can share meaningful insights through charts, KPIs, filters, and drill-down analysis, while business stakeholders can independently explore the data according to their requirements. This creates better collaboration, improves understanding, and supports faster evidence-based decision-making.